Movesense Flash Sensor Application Note

1. Introduction

In many uses there is a need to collect physiological and biomechanical data, such as the user's ECG, heart rate or movement information, for long periods of time without a continuous connection to a host device. Movesense Flash solves this need by offering an extended data storage capacity that enables standalone ECG and motion data recording without any user interaction and connection to a Bluetooth receiving device.

With its 128MB internal data storage memory, Movesense Flash can record up to 67h of single-channel ECG data at 125Hz sample rate (tested with device firmware 2.1.5), or more than 7 days of daily activity (3D acceleration at 26Hz sampling rate).

The recorded data files are downloaded from the sensor memory for post-analysis via Bluetooth. The Bluetooth transmission can be switched on and off with software as needed.

These application notes are intended for product owners and developers to compare Movesense Flash with other Movesense sensor models and evaluate the suitability of the device for their needs.

This application note is intended for developers and product owners for comparing Movesense Flash sensor with other Movesense sensor models, for evaluating its suitability for their needs, and for providing guidance for actual development projects based on the device.

2. Example use cases

The Movesense Flash sensor can be used for example for:

Long-term recording of HRV: Movesense Flash memory capacity is sufficient for over 3000 hours of R-R interval data (HR=90 bpm, 8 bytes / R-R interval). The battery will last about 900 hours in R-R recording without Bluetooth transmission.

Collecting large amount of raw data for AI modeling: For example, the device can store about 19 hours of IMU9 data at 52 Hz sampling rate before the memory is full.

Long-term ECG recording for well-being applications and research: Movesense Flash is not certified as a medical device but, with its ECG bandwidth of 0.5-40Hz, it enables accurate, long-term, standalone recording of single-channel ECG for non-medical applications.

Recording ECG and/or movement data underwater: Since water attenuates high-frequency radio signals very effectively, collecting movement, heart rate and ECG data underwater requires recording them directly with the measuring device. Movesense Flash is waterproof for swimming and diving at typical pool depths, making it suitable for collecting data in water. However, please note that due to the electrical conductivity of water, measuring the ECG in water requires electrodes specially designed for underwater use or protection of the electrodes from water.

3. Movesense sensor hardware variants

There are several variants of Movesense sensors available for developers. Table 1 below lists the different sensor types and some of their features, including a comparison to the Movesense Flash sensor:

Sensor model	HWCONFIG ID	Data memory	1-Wire master	Remarks
Movesense HR2	SS2	384 kB EEPROM	NO	
Movesense HR+	SS2	384 kB EEPROM	YES	
Movesense MD	SS2	384 kB EEPROM	NO	Class IIa medical device (MDR)
Movesense Flash	SS2_NAND	128 MB FLASH	NO	

Table 1. Movesense sensor hardware variants as of October 2023.

Note: Movesense Flash sensor has a different hardware configuration, HWCONFIG ID, compared to the other Movesense sensor models. Therefore, Movesense Flash sensor is not compatible with the firmware binaries intended for the other sensor hardware variants. A dedicated binary build with the Movesense Flash specific HWCONFIG needs to be compiled.

4. Maximum recording duration with different data combinations

The available data storage duration naturally depends on the recording data rate. Table 2 shows examples of the maximum recording time achievable with different data source and sample rate combinations. The listed recording times were evaluated with sensor firmware 2.1.5 and using a 32-bit sample size. Depending on the exact use case, a 16-bit or in some cases even an 8-bit sample size may be sufficient for storage and analysis, in which case the maximum achievable recording times can be twice or four times the ones listed in the table.

Recorded data	Maximal recording duration
ECG 125Hz	67.5h
ECG 256Hz	33h
ECG 512Hz	16.5h
ECG 200Hz + ACC3 26Hz	29h
IMU9 52Hz	18.5h
IMU9 52Hz + ECG 125Hz	14.5h
IMU9 52Hz + ECG 256Hz	12h
ACC3 26Hz	96h

Table 2. Recording time from empty to full memory with different data combinations

5. Movesense Flash memory characteristics and device operation

5.1. General

The key feature, and the main difference of the Movesense Flash sensor compared to other Movesense sensor models is the larger data storage memory. The 128MB flash memory is organized as a file system, instead of a ring buffer used in the sensors with 384kB EEPROM memory. This has the following implications:

- When the filesystem is full, incoming data can no longer be written (vs. in the case of EEPROM, where new data overwrites old ones).
- The minimum size of a single log (=file) is 128 kB and the size grows in chunks of 128 kB, i.e., the file size is reported as multiples of ~128 kB.
- The file system latency increases as the number of individual log files created increases. When storing data, there is a file system related delay after every 7 logs created, typically lasting from 300-400ms (memory empty to half-full) to 2000ms (memory nearly full). Even if accounted for, this can cause buffer overflow (on firmware versions 2.1.x) or data loss (version 2.2 and above). The effect is similar to the one described below, when the memory is completely full.
- When file system becomes almost full, some file system operations' response time increases further, and the processing delay will increase to about 2s.
- The file system only supports one (1) user at a time. As a result, simultaneous writing and reading is not possible. However, *GET /Mem/Logbook/Entries* can be called while writing at the same time.
- The file system implementation reserves much more RAM than the EEPROM DataLogger implementation, resulting in much less heap RAM available for the rest of the code.
- The flash memory uses more energy during both active operation, idling and even when not in use. Compared to the other Movesense sensor models, the overall current consumption increases by approximately 15μA when IDLE and additional ~100μA when recording continuous fast data such as ECG.

5.2. Effects of file system response time when the memory gets full

Due to the memory and the associated file system structure, the flash memory operations slow down when the memory becomes full. The memory block write process takes more time to complete, and depending on the incoming data rate, writing of the previous block might not be completed before the next one is already available.

Due to the limited size of data buffers available, in such case any pending data frame may end up being discarded, and this will cause data gaps lasting from 300ms to 2000ms. For example, when recording ECG 125Hz, the first such file system related delay (approximately 400ms) comes when the memory is half-full.

The effect depends on the recording data rate and other tasks the sensor is doing while recording data. The data packet time stamps always indicate the timewise integrity of the stored data.

As an example, recording raw IMU9 data with two different sample rates:

- Sample rate 52Hz: The first accelerometer FIFO overflow (= data frame loss) happens when about 95MB of memory is used.
- Sample rate 208Hz: The first accelerometer FIFO overflow happens when about 50MB of memory is used.

In both cases, when the memory is almost full (~125 MB used), the file system operations related delays will increase to a level where the application thread can no longer empty the accelerometer data from the Whiteboard queue (at default length), resulting ASSERT in Whiteboard.cpp, and a consequent sensor reset may occur.

Therefore, it is highly recommended that when designing the end application, unnecessarily high data rates are avoided, especially at increasing flash memory fill levels. It is also important that the application specific behavior is carefully evaluated and profiled.

Figures 1-4 below show examples of real-life data gap events with certain sampling data combinations. In the graphs, the x-axis is the memory fill rate in percent and the y-axis is the data gap event length in milliseconds. Each dot represents one gap event. For each area indicated by brackets, the total length of the gap events has been calculated as a percentage of the total amount of stored data at the respective memory fill level.

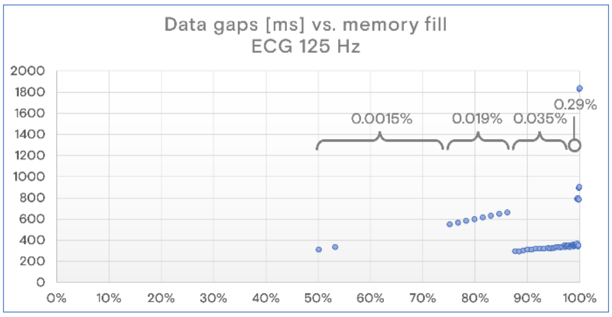

Figure 1. *Profile of the data gap events when filling an empty memory with 125Hz ECG recording. A total of 54 gaps, whose duration varies between 300ms and 2000ms. The first event happens when the memory is approximately 50% full.*

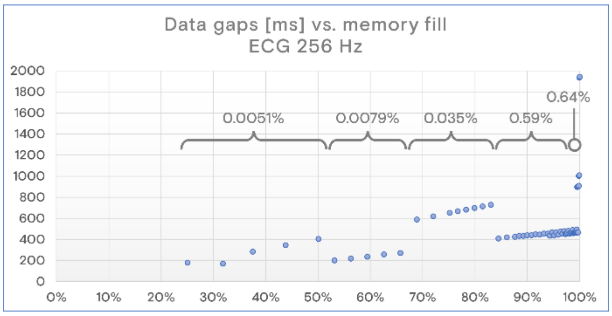

Figure 2. *Profile of the data gap events when filling an empty memory with 256Hz ECG recording. A total of 64 gaps, whose duration varies between 200ms and 2000ms. The first event occurs when the memory is approximately 25% full.*

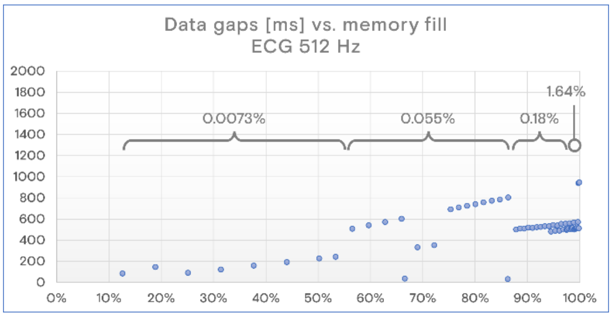

Figure 3. *Data gaps events with 512Hz ECG recording. 66 gaps of 35-1000ms.*

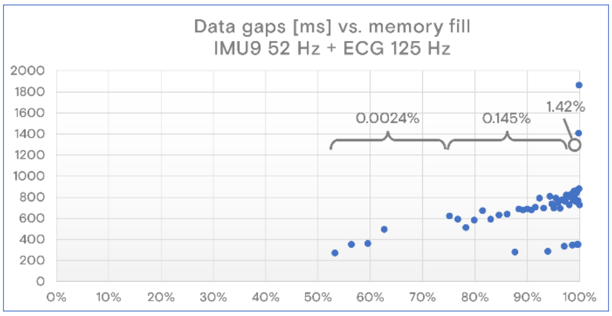

Figure 4. *Data gaps events with IMU9 52Hz + 125Hz ECG recording. 55 gap events of 300-2000ms.*

5.3. Maximum storage data rate

Table 3 presents the number of data gap events during a 2-min test recording with an initially empty memory using different data combinations and sample rates. When using max 416Hz sampling rate for the inertial measurement and any of the available ECG sample rates, Movesense Flash can process the incoming data stream when memory fill rate dependent file system delays are not encountered.

We always recommend testing your own implementation with different data transfer speeds, filling the entire data storage capacity. In general, in terms of memory capacity, power consumption, and user experience it is advisable to use the lowest data rate that produces a sufficient amount of data for your use case.

IMU9 Sample rate [Hz]	ECG Sample rate [Hz]	FIFO IMU9 fails	FIFO ECG fails	TOTAL fails	Data rate ok?
416	-	0	-	0	Ok
833	-	4	-	4	Not recommended
1666	-	8	-	8	Not recommended
416	125	0	0	0	Ok
416	200	0	0	0	Ok
416	256	0	0	0	Ok
416	512	0	0	0	Not recommended
833	125	6	0	6	Not recommended
833	200	5	3	8	Not recommended

Table 3. Recommendations for using high data rates when storing data with Movesense Flash.

5.4. Effects of low battery voltage

Flash memory is more sensitive to low voltage events, especially during the write operations, compared to EEPROM memory used in the other Movesense sensor variants.

Therefore, Movesense Flash sensor implements an additional battery voltage monitoring functionality. In software, the output of this monitoring can be read by subscribing to the *"BatteryStatus"*-state (StateID=1) in */System/States/{StateID}* -API.

The *DataLoggerService* subscribes to this API when it is recording data and stops logging automatically before the remaining battery capacity decreases to a level where the flash memory operation would be at risk.

Note: We recommended to *SUBSCRIBE* (or *GET*) this value in your firmware to determine if low voltage flag has been triggered. Typically, this may happen in situations with high current consumption, such as when writing to flash memory or during heavy BLE transfers.

Note: If the recording ends due to a low battery, it may be necessary to replace the battery before downloading the recorded data to ensure successful download. If you are developing your own mobile app that works with the Movesense Flash, we recommend adding a battery status check to the application in relevant moments.

6. Power consumption

The Movesense Flash sensor is powered by a size CR2025 primary coin cell battery. The total capacity available in a high-quality Maxell CR2025 battery is approximately 400mWh. Approximately 70% of the nominal battery capacity is usable.

Whereas the Movesense Flash sensor is designed to be power-optimized, the flash memory access operations, as well as the continuous high data rate during the wireless transfer of the memory data does increase the power consumption, compared to other Movesense sensor variants.

Recorded data	Number of full recording + download cycles (recording time for one cycle)	Total recording time with one battery
ECG 256Hz	6 cycles (33h)	200h
ECG 125Hz	4 cycles (67.5h)	270h 300h with shorter recording cycles
ACC3 26Hz	5 cycles (96h)	480h

Table 4. Typical operating time with a single Maxell CR2025 coin cell battery. Recording times indicated with sensor firmware 2.1.5 using 32-bit sample size.

A full memory download alone consumes approximately 5% of the usable battery capacity (~15mWh).

7. Download speed over Bluetooth

Data download speed over Bluetooth connection depends on the capabilities of the used Bluetooth host device (usually a phone or a tablet computer) and the MDS library version used. MDS libraries 1.6.8 and later support the Fast Logbook Data Fetch functionality. We recommend using MDS 3.15 or newer on the mobile side to ensure a smooth JSON conversion of the downloaded data. Note: this also requires sensor firmware 2.2. or higher.

Measured download speeds using an Android terminal:

- Default 1M PHY BLE: 54 kB/s
- 2M PHY mode: 75 kB/s, full memory download in under 40min.

8. Known limitations

- Really fast data rates such as IMU9 with sample rate 833Hz or higher cannot be stored on Flash memory without data loss. See maximum data rates in Table 3.
- The data structure of Movesense device lib 2.1.x and earlier requires the whole data to be in the mobile device's memory for SBEM-JSON conversions. This can cause mobile app crash due to "Out of Memory" error. The data format has been re-designed in the firmware release 2.2, but until then JSON downloads require shorter recordings or a mobile phone with >8GB of RAM. Alternatively, a raw SBEM format download (see DataLoggerSample –application for example) can be used.

- Both Flash memory writing and fast data transfer momentarily consume large amounts of power. Therefore, the whole battery capacity is not available for these operations. Typically, the battery needs to be replaced at around 30% level.

9. Further reading

Movesense developer documentation: https://www.movesense.com/docs/

10. Copyright